PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)

PROJECT REPORT

TOPIC- MONGODB STUDENT DATABASE OPERATIONS (REVISED DATASET)

IBM

SUBMITTED TO- MR. Harendra Singh Rajpoot
Submitted by- Yuvraj Singh
BCADS27
Roll no. - 1250258515
3rd semester
Babu Banarasi Das University
Lucknow, U.P

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor Mr. Harendra Singh Rajpoot, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to IBM, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. This revised version of the project replaces the original dataset with a new set of 51 student records (Roll No. 201-251) while keeping the exact same structure, operations and query logic as the original report. A "students" collection is used to store student records containing roll number, name, age, department and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as $gt, $lt, $eq, $gte, $ne, $and, $or and $not, with all outputs recomputed against the new dataset.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 51 students (Roll No. 201 to 251), and demonstrates the standard database operations - insertion, updation, deletion and querying - that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a database created for this purpose. The complete methodology, along with the exact code executed and the output produced by MongoDB at each step, is given below.

1. Insert One

Inserts a single student document into the students collection.

Code:

```
db.students.insertOne({
 rollNo: 201,
 name: "Aarav",
 age: 21,
 department: "CSE",
 marks: 64
})
```

2. Insert Many

Inserts multiple student documents (roll numbers 202 to 251) into the collection at the same time.

Code:

```
db.students.insertMany([
{ rollNo: 202, name: "Ishita", age: 20, department: "IT", marks: 96 },
{ rollNo: 203, name: "Kabir", age: 22, department: "CSE", marks: 63 },
{ rollNo: 204, name: "Meera", age: 19, department: "ECE", marks: 99 },
{ rollNo: 205, name: "Rohan", age: 19, department: "ME", marks: 73 },
{ rollNo: 206, name: "Tanvi", age: 23, department: "CSE", marks: 91 },
{ rollNo: 207, name: "Dev", age: 19, department: "IT", marks: 94 },
{ rollNo: 208, name: "Ananya", age: 21, department: "ECE", marks: 87 },
{ rollNo: 209, name: "Vivaan", age: 23, department: "CSE", marks: 80 },
{ rollNo: 210, name: "Sana", age: 19, department: "IT", marks: 89 },
{ rollNo: 211, name: "Aryan", age: 23, department: "ME", marks: 97 },
{ rollNo: 212, name: "Diya", age: 20, department: "CSE", marks: 89 },
{ rollNo: 213, name: "Krishna", age: 19, department: "ECE", marks: 83 },
{ rollNo: 214, name: "Nidhi", age: 19, department: "IT", marks: 79 },
{ rollNo: 215, name: "Sameer", age: 22, department: "CSE", marks: 75 },
{ rollNo: 216, name: "Pallak", age: 22, department: "ECE", marks: 71 },
{ rollNo: 217, name: "Rudra", age: 19, department: "ME", marks: 75 },
{ rollNo: 218, name: "Kriti", age: 20, department: "CSE", marks: 65 },
{ rollNo: 219, name: "Aditya", age: 19, department: "IT", marks: 96 },
{ rollNo: 220, name: "Zara", age: 23, department: "ECE", marks: 79 },
{ rollNo: 221, name: "Yash", age: 22, department: "CSE", marks: 93 },
{ rollNo: 222, name: "Lavanya", age: 19, department: "IT", marks: 91 },
{ rollNo: 223, name: "Om", age: 23, department: "ME", marks: 81 },
{ rollNo: 224, name: "Trisha", age: 19, department: "CSE", marks: 88 },
{ rollNo: 225, name: "Vihaan", age: 20, department: "ECE", marks: 78 },
{ rollNo: 226, name: "Radhika", age: 23, department: "IT", marks: 98 },
{ rollNo: 227, name: "Kunal", age: 19, department: "CSE", marks: 64 },
{ rollNo: 228, name: "Sneha", age: 23, department: "ECE", marks: 67 },
{ rollNo: 229, name: "Ishaan", age: 23, department: "ME", marks: 92 },
{ rollNo: 230, name: "Aditi", age: 22, department: "CSE", marks: 86 },
{ rollNo: 231, name: "Reyansh", age: 19, department: "IT", marks: 70 },
{ rollNo: 232, name: "Bhavya", age: 20, department: "ECE", marks: 81 },
{ rollNo: 233, name: "Siddharth", age: 19, department: "CSE", marks: 69 },
{ rollNo: 234, name: "Charu", age: 23, department: "IT", marks: 91 },
{ rollNo: 235, name: "Advait", age: 20, department: "ME", marks: 86 },
{ rollNo: 236, name: "Mahi", age: 21, department: "CSE", marks: 62 },
{ rollNo: 237, name: "Parth", age: 22, department: "ECE", marks: 64 },
{ rollNo: 238, name: "Ira", age: 20, department: "IT", marks: 95 },
{ rollNo: 239, name: "Naman", age: 23, department: "CSE", marks: 96 },
{ rollNo: 240, name: "Kiara", age: 19, department: "ECE", marks: 80 },
{ rollNo: 241, name: "Dhruv", age: 23, department: "ME", marks: 81 },
{ rollNo: 242, name: "Riya", age: 21, department: "CSE", marks: 82 },
{ rollNo: 243, name: "Shaurya", age: 23, department: "IT", marks: 98 },
{ rollNo: 244, name: "Anvi", age: 20, department: "ECE", marks: 91 },
{ rollNo: 245, name: "Veer", age: 19, department: "CSE", marks: 97 },
{ rollNo: 246, name: "Myra", age: 23, department: "IT", marks: 89 },
{ rollNo: 247, name: "Arnav", age: 23, department: "ECE", marks: 64 },
{ rollNo: 248, name: "Saanvi", age: 20, department: "ME", marks: 65 },
{ rollNo: 249, name: "Kian", age: 21, department: "CSE", marks: 77 },
{ rollNo: 250, name: "Navya", age: 19, department: "IT", marks: 90 },
{ rollNo: 251, name: "Tanish", age: 23, department: "ECE", marks: 64 }])
```

3. Update One

Updates the first document that matches the given condition - here, the marks of roll number 201 are updated from 64 to 72.

Code:

```
db.students.updateOne(
 { rollNo: 201 },
 { $set: { marks: 72 } }
)
```

Output:

```
{
 acknowledged: true,
 matchedCount: 1,
 modifiedCount: 1
}
```

4. Update Many

Updates all documents that match the given condition - here, every student in the CSE department is given a new status field set to "Active".

Code:

```
db.students.updateMany(
 { department: "CSE" },
 { $set: { status: "Active" } }
)
```

Output:

```
{
 acknowledged: true,
 matchedCount: 17,
 modifiedCount: 17
}
```

5. Delete One

Deletes the first document that matches the given condition - here, the record of roll number 251 (Tanish) is removed.

Code:

```
db.students.deleteOne({ rollNo: 251 })
```

Output:

```
{
 acknowledged: true,
 deletedCount: 1
}
```

6. Delete Many

Deletes all documents that match the given condition - here, every student with marks less than 40 is removed.

Code:

```
db.students.deleteMany({ marks: { $lt: 40 } })
```

Output:

```
{
 acknowledged: true,
 deletedCount: 0
}
```

7. Query Operations

A series of find() queries were run using MongoDB's comparison and logical operators to filter and retrieve student records from the final 50-record collection.

Marks greater than 80:

```
db.students.find({ marks: { $gt: 80 } })
```

Roll No	Name	Age	Dept	Marks
202	Ishita	20	IT	96
204	Meera	19	ECE	99
206	Tanvi	23	CSE	91
207	Dev	19	IT	94
208	Ananya	21	ECE	87
210	Sana	19	IT	89
211	Aryan	23	ME	97
212	Diya	20	CSE	89
213	Krishna	19	ECE	83
219	Aditya	19	IT	96
221	Yash	22	CSE	93
222	Lavanya	19	IT	91
223	Om	23	ME	81
224	Trisha	19	CSE	88
226	Radhika	23	IT	98
229	Ishaan	23	ME	92
230	Aditi	22	CSE	86
232	Bhavya	20	ECE	81
234	Charu	23	IT	91
235	Advait	20	ME	86
238	Ira	20	IT	95
239	Naman	23	CSE	96
241	Dhruv	23	ME	81
242	Riya	21	CSE	82
243	Shaurya	23	IT	98
244	Anvi	20	ECE	91
245	Veer	19	CSE	97
246	Myra	23	IT	89
250	Navya	19	IT	90

Total matched: 29

Marks less than 80:

```
db.students.find({ marks: { $lt: 80 } })
```

Roll No	Name	Age	Dept	Marks
201	Aarav	21	CSE	72

203	Kabir	22	CSE	63
205	Rohan	19	ME	73
214	Nidhi	19	IT	79
215	Sameer	22	CSE	75
216	Pallak	22	ECE	71
217	Rudra	19	ME	75
218	Kriti	20	CSE	65
220	Zara	23	ECE	79
225	Vihaan	20	ECE	78
227	Kunal	19	CSE	64
228	Sneha	23	ECE	67
231	Reyansh	19	IT	70
233	Siddharth	19	CSE	69
236	Mahi	21	CSE	62
237	Parth	22	ECE	64
247	Arnav	23	ECE	64
248	Saanvi	20	ME	65
249	Kian	21	CSE	77

Total matched: 19

Marks equal to 80:

```
db.students.find({ marks: { $eq: 80 } })
```

Roll No	Name	Age	Dept	Marks
209	Vivaan	23	CSE	80
240	Kiara	19	ECE	80

Total matched: 2

Marks greater than or equal to 80:

```
db.students.find({ marks: { $gte: 80 } })
```

Roll No	Name	Age	Dept	Marks
202	Ishita	20	IT	96
204	Meera	19	ECE	99
206	Tanvi	23	CSE	91
207	Dev	19	IT	94
208	Ananya	21	ECE	87
209	Vivaan	23	CSE	80
210	Sana	19	IT	89
211	Aryan	23	ME	97
212	Diya	20	CSE	89
213	Krishna	19	ECE	83
219	Aditya	19	IT	96
221	Yash	22	CSE	93

222	Lavanya	19	IT	91
223	Om	23	ME	81
224	Trisha	19	CSE	88
226	Radhika	23	IT	98
229	Ishaan	23	ME	92
230	Aditi	22	CSE	86
232	Bhavya	20	ECE	81
234	Charu	23	IT	91
235	Advait	20	ME	86
238	Ira	20	IT	95
239	Naman	23	CSE	96
240	Kiara	19	ECE	80
241	Dhruv	23	ME	81
242	Riya	21	CSE	82
243	Shaurya	23	IT	98
244	Anvi	20	ECE	91
245	Veer	19	CSE	97
246	Myra	23	IT	89
250	Navya	19	IT	90

Total matched: 31

Marks not equal to 80:

```
db.students.find({ marks: { $ne: 80 } })
```

Total matched: 48 (all students except the 2 with marks exactly 80).

Marks greater than 70 AND age less than 23:

```
db.students.find({
 $and: [
 { marks: { $gt: 70 } },
 { age: { $lt: 23 } }
 ]
})
```

Roll No	Name	Age	Dept	Marks
201	Aarav	21	CSE	72
202	Ishita	20	IT	96
204	Meera	19	ECE	99
205	Rohan	19	ME	73
207	Dev	19	IT	94
208	Ananya	21	ECE	87
210	Sana	19	IT	89
212	Diya	20	CSE	89
213	Krishna	19	ECE	83
214	Nidhi	19	IT	79
215	Sameer	22	CSE	75

216	Pallak	22	ECE	71
217	Rudra	19	ME	75
219	Aditya	19	IT	96
221	Yash	22	CSE	93
222	Lavanya	19	IT	91
224	Trisha	19	CSE	88
225	Vihaan	20	ECE	78
230	Aditi	22	CSE	86
232	Bhavya	20	ECE	81
235	Advait	20	ME	86
238	Ira	20	IT	95
240	Kiara	19	ECE	80
242	Riya	21	CSE	82
244	Anvi	20	ECE	91
245	Veer	19	CSE	97
249	Kian	21	CSE	77
250	Navya	19	IT	90

Total matched: 28

Department is CSE OR ECE:

```
db.students.find({
 $or: [
 { department: "CSE" },
 { department: "ECE" }
 ]
})
```

CSE:

Roll No	Name	Age	Dept	Marks
201	Aarav	21	CSE	72
203	Kabir	22	CSE	63
206	Tanvi	23	CSE	91
209	Vivaan	23	CSE	80
212	Diya	20	CSE	89
215	Sameer	22	CSE	75
218	Kriti	20	CSE	65
221	Yash	22	CSE	93
224	Trisha	19	CSE	88
227	Kunal	19	CSE	64
230	Aditi	22	CSE	86
233	Siddharth	19	CSE	69
236	Mahi	21	CSE	62
239	Naman	23	CSE	96
242	Riya	21	CSE	82
245	Veer	19	CSE	97

249	Kian	21	CSE	77

ECE:

Roll No	Name	Age	Dept	Marks
204	Meera	19	ECE	99
208	Ananya	21	ECE	87
213	Krishna	19	ECE	83
216	Pallak	22	ECE	71
220	Zara	23	ECE	79
225	Vihaan	20	ECE	78
228	Sneha	23	ECE	67
232	Bhavya	20	ECE	81
237	Parth	22	ECE	64
240	Kiara	19	ECE	80
244	Anvi	20	ECE	91
247	Arnav	23	ECE	64

Total matched: 29 (CSE: 17, ECE: 12)

Marks NOT greater than 80:

```
db.students.find({ marks: { $not: { $gt: 80 } } })
```

Roll No	Name	Age	Dept	Marks
201	Aarav	21	CSE	72
203	Kabir	22	CSE	63
205	Rohan	19	ME	73
209	Vivaan	23	CSE	80
214	Nidhi	19	IT	79
215	Sameer	22	CSE	75
216	Pallak	22	ECE	71
217	Rudra	19	ME	75
218	Kriti	20	CSE	65
220	Zara	23	ECE	79
225	Vihaan	20	ECE	78
227	Kunal	19	CSE	64
228	Sneha	23	ECE	67
231	Reyansh	19	IT	70
233	Siddharth	19	CSE	69
236	Mahi	21	CSE	62
237	Parth	22	ECE	64
240	Kiara	19	ECE	80
247	Arnav	23	ECE	64
248	Saanvi	20	ME	65
249	Kian	21	CSE	77

Total matched: 21

8. MongoDB Operator Summary

Operator	Meaning	Example
$gt	Greater than	marks: { $gt: 80 }
$lt	Less than	marks: { $lt: 80 }
$eq	Equal to	marks: { $eq: 80 }
$gte	Greater than or equal to	marks: { $gte: 80 }
$ne	Not equal to	marks: { $ne: 80 }
$and	All conditions must be true	marks > 70 AND age < 23
$or	At least one condition must be true	CSE OR ECE
$not	Negates a condition	NOT (marks > 80)

Total dataset: 51 students inserted (Roll No. 201-251); 50 students remain after deleteOne removed roll number 251.

Implementation Details

- The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh).
- A single collection named "students" is used, with fields: rollNo, name, age, department and marks.
- insertOne() and insertMany() are used to populate the collection with 51 student records (Roll No. 201-251).
- updateOne() and updateMany() with the $set operator are used to modify existing fields and add new ones (e.g. status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.
- find() is combined with comparison operators ($gt, $lt, $eq, $gte, $ne) and logical operators ($and, $or, $not) to retrieve specific subsets of student data.
- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the students collection with the new dataset. The insertOne and insertMany operations populated the collection with 51 student records (Roll No. 201-251). The updateOne operation successfully changed roll number 201's marks from 64 to 72, while updateMany correctly modified all 17 CSE-department documents to add an "Active" status. The deleteOne operation removed the record of roll number 251, and deleteMany found no records matching its filter (marks less than 40), correctly reporting a deletedCount of 0. The query operations correctly filtered the remaining 50 students by marks and department using comparison and logical operators, demonstrating that MongoDB's query engine returns accurate, condition-based results across the new, differently valued dataset.

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. Re-running the exact same set of operations on a completely new dataset, and obtaining correctly recomputed outputs, further confirms that the approach is generalizable and not tied to any one specific set of records.

Future Scope

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.
- Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

References

- MongoDB Official Documentation: https://www.mongodb.com/docs/
- MongoDB CRUD Operations: https://www.mongodb.com/docs/manual/crud/
- MongoDB Query Operators: https://www.mongodb.com/docs/manual/reference/operator/query/
- mongosh (MongoDB Shell) Documentation: https://www.mongodb.com/docs/mongodb-shell/

THANK YOU!